Virtual Programming Fundamentals Internship
in Ahmedabad

Project Planning and Scope Definition

A programming fundamentals module for non-programmers

Author: Meghana Krishnakumar Nair
Week 1 of 4 | Offer received 2 November 2026
Internship period: 2 November 2026 to 30 November 2026
Report prepared on 4 November 2026

This plan shows how a complete beginner can go from "never coded" to "I built something useful" in 30 teaching hours, with no prior knowledge and no software to install.

Table of contents

Use the list below to find each section quickly. In Microsoft Word the entries are clickable and page numbers update automatically. In this file the contents are a live field, so right-click the list and choose Update Field after opening in Word to refresh the numbers.

Right-click and choose Update Field in Word to populate page numbers after opening.

Executive summary 3
1 Overview and purpose 4
2 Target audience 5
3 Research summary 6
4 Educational objectives and learning outcomes 7
5 Key content areas and curriculum structure 9
6 Instructional methodologies 11
7 Assessment techniques 12
8 Resource requirements and allocation 14
9 Timeline, milestones and deadlines 15
10 Risks, assumptions and mitigations 16
11 Success metrics and evaluation 17
12 Conclusion and next steps 18
Glossary 18
References 19
Appendix: assumptions and decisions log 20

Executive summary

This document plans a short, friendly programming course for adults and students who have never written a line of code. The course teaches Python, a language that reads almost like clear English, using only a web browser so learners do not need to install anything. It runs for ten modules and thirty teaching hours. By the end, each learner will have built one small, useful program of their own, such as a menu tool or a file organiser, and will be able to explain how it works in plain words.

The plan is written for non-technical readers. Every new word is explained the first time it appears, and each section gives the reasoning behind the decision, not just the decision itself. We start with thinking in steps, then learn to store information, make choices, repeat actions, group steps into reusable pieces, organise lists of data, work with files, and finish by reading error messages calmly and fixing them. Teaching uses short explanations, many small practice tasks, pair work, and quick feedback, because beginners learn best by typing and by seeing how a program behaves, not by watching long talks.

Assessment is gentle and frequent. Short quizzes, exit tickets, and small exercises build confidence, while the final project shows real understanding. Success for the programme means most learners finish the project and can describe what they did and what they would do next. The timeline in Section 9 is a planned project, starting on the baseline date of 5 November 2026 (planned), with six phases, six milestones, and 120 planned hours across five roles. Costs are modest because free tools and free reading materials are used wherever possible.

1 Overview and purpose

This internship offer was received on 2 November 2026 and the internship runs from 2 November 2026 to 30 November 2026 (four weeks, four tasks). This Week 1 report was prepared on 4 November 2026. It plans the whole module before any teaching starts. The question it answers is simple: how do we teach programming to people who think of themselves as “not technical”, in a way that feels welcoming, practical, and achievable? Everything in this plan — the order of modules, the choice of language, the session shape, the assessment, and the budget — follows from that question.

A few definitions will help. Programming means writing instructions that a computer follows step by step. A program is the set of those instructions. A module is one teaching unit, roughly three hours long, that builds on the one before it. The capstone is the final mini-project, where learners combine what they have learned into one finished program. We use the term programme for the whole course, and module for each part inside it. Keeping the language plain matters, because learners who feel excluded by jargon stop trying early.

Why Python

We teach in Python. Three reasons support this choice. First, Python reads like a well-written list of steps. A command that shows a message looks like a normal English word rather than a symbol, which lowers the shock of seeing code for the first time [11][12]. Second, Python runs directly in a web browser through Google Colab or a browser editor, so learners can start in the first session without installing anything, which matters for shared computers, Chromebooks, and learners with limited disk space. Third, Python is common enough that learners can find help later in free books and community pages [2][3].

We considered two main alternatives. Scratch, a drag-and-drop language, is excellent for younger children because removing typing removes fear, but for adults who want text skills that transfer to jobs and further study, it can feel like a toy and does not teach the habit of writing real statements [11]. JavaScript is also popular, but its punctuation rules and browser setup create more early errors, and some of its ideas (like strict symbol use) reach learners before they have confidence. Python gives us readable statements, quick feedback, and real tasks such as reading lists of names or writing simple files from the first week. That balance of ease and usefulness is why it is our choice.

Scope in one view

- In scope: one 30-hour module for complete beginners, taught online, in Python, with 10 modules and a final mini-project.
- In scope: lesson plans, slides, handouts, assessment tasks, and a pilot check before a wider launch.

- Out of scope: advanced topics such as databases, web development, or object-oriented design. These may be a later course.
- Assumption: learners have access to a browser and a stable enough internet connection to use Colab.

The rest of the document details who the learners are, what research says about teaching them well, exactly what they will learn, how each session runs, how we know they learned, what people and money are needed, when milestones fall, what could go wrong, and how we will judge whether the programme itself worked.

2 Target audience

Our learners are adults or undergraduate students with no programming background. They may be good at other subjects, they may use computers every day for documents or messages, but they have never written a program. They are not opting for a heavy theory course; they want a clear, useful skill they can explain to friends and family. Motivation is often practical — to automate a tedious job, to understand a work tool, or to see if computing is for them — rather than abstract. Anxiety is common at the start, especially after their first error message, so early reassurance is part of the teaching plan, not an extra.

Three personas help us organise decisions about pace, examples, and support. They are not stereotypes, but useful planning tools that guide choices about wording, workload, and pairing. Prior knowledge across all three is similar (everyday computer use, no code), but what they want and what worries them differ enough to shape the design.

Persona A: the graduate stuck with repetitive work

Neha, 26, works in administration. Her team copies data between files by hand each week. She wants a small program that sorts or counts for her. Her strength is persistence; her barrier is time and fear of breaking something. She learns best by seeing a real task done first, then copying the pattern and changing the names and numbers. She responds to worked examples, calm error reading, and clear "what you did well" feedback. Accessibility needs: captions on videos, handouts she can read offline, and sessions at a predictable time with notes afterwards.

Persona B: the undergraduate testing the water

Arjun, 19, studies business. He has tried a video once and got stuck on the first error. He is curious but quick to compare himself with classmates who seem faster. His strength is curiosity; his barrier is low patience when typing feels slow. He learns best in pairs, where he can talk through a problem before typing, and through short challenges with visible progress. He will stick with the course if he sees points, badges, or a finished artifact that shows he belongs. Accessibility needs: extra keyboard shortcuts, high-contrast slides, and permission to use speech-to-text where relevant.

Persona C: the career-changer with high motivation

Fatima, 32, teaches at a school and wants to move into data support work. She is highly motivated and organised, but worries that she is "too late" to learn. Her strength is method; her barrier is perfectionism that can turn a small bug into a big setback. She learns best by predicting what code will do, then testing, and by keeping a short error log to spot patterns. She needs visible milestones and a rubric that rewards progress, not only a perfect final program. Accessibility needs: plain-English glossary, dyslexia-friendly font options, and recordings she can replay.

Persona	Main motivation	Biggest barrier	What helps most
A. Graduate, repetitive work	Automate a real job	Fear of breaking things	Worked examples, copy-then-change
B. Undergraduate, curious	Belonging and proof	Low patience with syntax	Pair work, visible progress
C. Career-changer, organised	New direction, proof	Perfectionism after errors	Predict–test, error logs, rubric

Across all three, accessibility is a design requirement, not an afterthought. We plan captioned videos, text transcripts, keyboard-friendly editors, slides that pass colour contrast checks, and handouts in a large, clear typeface. We also use colour together with labels or icons, never colour alone, so that meaning is clear for colour-

blind learners. Sessions include regular breaks, and pair work is arranged so learners can rotate roles and avoid being stuck with a much faster partner every week.

Figure 2: Learner journey — from “never coded” to “I built something”



Small wins early, harder challenges later. Confidence is part of the curriculum, not an accident.

Figure 2: Learner journey from “never coded” to “built something”. *The course gives small, visible wins early so confidence grows before harder ideas arrive.*

Image description: A horizontal journey with six boxes: Never coded, Ran a first program, Predicted and ran code, Wrote small programs, Debugged with help, Built the capstone. Arrows show forward progress.

3 Research summary: challenges non-programmers face and what works

Good planning starts with honest evidence. Public beginner courses and teaching studies point in the same direction: the ideas in programming are not especially hard, but the way we introduce them can make them feel hard. Learners struggle with unfamiliar words, with holding many steps in mind at once, with mistaking a variable for a box, and with the emotional sting of an error message. They also lose motivation fast if the first task feels pointless or if they must watch passively for too long. The strategies below respond to those findings and are used throughout Sections 5 to 7.

What typical beginner courses do (and why we follow the same spine)

CS50x, Python for Everybody, Automate the Boring Stuff, and Code.org all follow a similar arc, even though their audiences differ [1][2][3][4]. CS50x moves from thinking without a computer to writing real programs, covering abstraction, algorithms, conditionals, loops, and functions in that order [1]. Python for Everybody teaches the smallest working program first, then adds variables, conditionals, and loops in short steps, on the principle that learners should have a success within minutes [2]. Automate the Boring Stuff uses the same idea for adults, teaching flow control early so learners can make a program that does a tedious job quickly, then building toward files and data [3]. Code.org places unplugged activities, where learners sort cards or trace steps without a screen, before block coding, which suggests that thinking in steps can be taught with ordinary objects before formal code [4].

Our module copies this spine because it is both logical and humane: think in steps on paper, run a tiny program early, add one new idea at a time, and finish by building something useful. The order reduces the number of new things a learner must hold in mind at once, which matters more than it first seems.

Challenge 1: cognitive overload

When a learner must listen, type, and debug at the same time, many new items compete for attention. Research on worked examples finds that novices learn more when they first study a fully worked example with the thinking made visible — the teacher shows each step and says why — before attempting a similar task [7][10]. Live coding, where the instructor thinks aloud while typing, serves the same purpose in a class. Our plan therefore uses a short worked example at the start of each session, then pairs learners on Parsons problems, where they order scrambled lines of code instead of writing from a blank page [5][6][8]. Parsons problems lower the load because the learner focuses on order and logic before worrying about punctuation. Only after ordering and predicting do learners write or change code themselves.

Challenge 2: syntax errors and the sting of failure

A syntax error means the computer did not understand the shape of a sentence, not that the learner is bad at thinking. Yet beginners often read it as failure and stop trying [7]. The plan tackles this by normalising errors from day one: the instructor deliberately makes and fixes one live, talks through the message in plain words, and keeps a class "error log" where useful fixes are collected. Learners first meet errors in code that is almost correct, so the task is to spot the mismatch rather than to invent a solution under pressure. File and data tasks in later modules are chosen because they give immediate, visible results, which rewards persistence [3].

Challenge 3: misconceptions

Common misunderstandings include treating a variable as a physical box, thinking a loop only means "repeat many times" without a stop condition, and assuming a computer follows the story they told rather than the steps they wrote. Unplugged sorting and tracing activities help here [4]: learners physically order steps for making tea, then trace a loop with their finger to see a counter change. In code, we ask learners to predict the output before running, write it down, then compare. The gap between prediction and result makes misconceptions visible without blame.

Challenge 4: motivation and belonging

Pair programming — two learners share one keyboard, one who types and one who talks through choices — is one of the most consistent findings for beginners. Reviews report higher confidence, better accuracy, and stronger conceptual understanding when novices work in pairs, compared with working alone under the same conditions [9]. We use pairs every session, with scheduled swaps and a simple rule: talk first, type second. Short, frequent formative checks, such as exit tickets and one-question quizzes, give early signs of trouble before a bigger task, and they make progress visible, which supports motivation. The final mini-project matters here: learners choose a task from their own life, such as counting words in an essay or organising a simple list, so the program has personal meaning [3].

In plain terms, our research summary tells us to sequence ideas carefully, use worked examples and ordering tasks before free writing, make errors normal, pair learners, give quick feedback, and end with a useful personal project. Sections 5 to 7 turn those findings into a timetable, a session shape, and an assessment plan.

4 Educational objectives and learning outcomes

Objectives say what the course intends to do. Learning outcomes say what learners will be able to do, in observable terms. We phrase each outcome with an action verb so that we can check it, and we map each one to a level of Bloom's taxonomy — a common framework described in the References notes — which moves from remembering and understanding toward applying, analysing, and creating. In practice, this means early outcomes focus on understanding and describing, middle outcomes on using ideas correctly, and later outcomes on combining ideas to build something new. The mapping table below shows where each outcome is taught and where it is checked, which is the bridge between Sections 5 and 7.

Outcomes are few on purpose. Seven outcomes can be taught well in thirty hours; twenty would be a wish list. Each one can be shown in a small task, a quiz, or the capstone, so learners and assessors share the same understanding of success.

Outcome	Bloom level	Learner will be able to (observable)	Assessed by
LO1	Understand	Describe the steps of a computer program in their own words	Quizzes, exit tickets
LO2	Understand	Explain what a short program does by reading its lines	Quizzes, small exercises
LO3	Apply	Write and run a small Python program that takes input and shows output	Small exercises, capstone
LO4	Apply	Use conditionals and loops	Small exercises, capstone

Outcome	Bloom level	Learner will be able to (observable)	Assessed by
		correctly to make choices and repeat actions	
LO5	Apply	Organise data in lists and dictionaries and use them in a program	Small exercises, capstone
LO6	Analyse	Read an error message, locate the line, and suggest a fix	Quizzes, small exercises, capstone
LO7	Create	Combine variables, conditionals, loops, functions, and files into a finished mini-project and describe it	Capstone

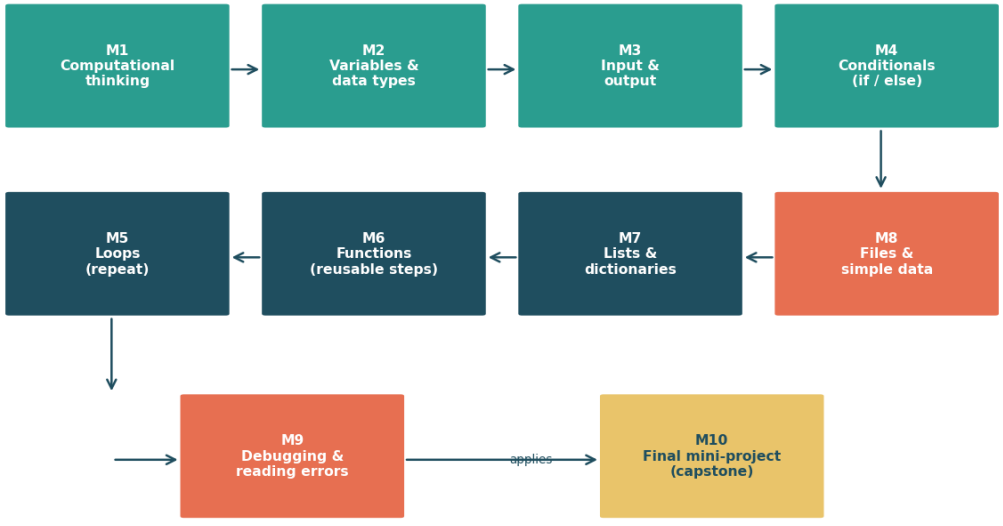
Outcome	Modules (where taught)	Assessments (where checked)	Why this matters
LO1	M1	Quizzes, Exit tickets	Sets the mindset: programs are step-by-step instructions
LO2	M2, M3	Quizzes, Small exercises	Reading code is the first real skill
LO3	M3, M6	Small exercises, Capstone	Writing and running is the core habit
LO4	M4, M5	Small exercises, Capstone	Choices and repeats are the heart of logic
LO5	M7, M8	Small exercises, Capstone	Organised data makes programs useful
LO6	M9 (in every module)	Quizzes, Small exercises, Capstone	Debugging is a life skill, not a side topic
LO7	M10 (uses M1–M9)	Capstone	Proof that ideas combine into something real

Read the second table row by row. LO1 is taught only in Module 1 but checked twice, which is right for an early idea. LO3 onward are checked in the capstone, so learners cannot pass by memorising alone. Every outcome appears in at least one module and at least one assessment, and most appear in two or three places, which guards against one bad day ruining a result. Figure 5 in Section 7 shows the same map as a picture, because the plan should be usable at a glance in a staff meeting.

5 Key content areas and curriculum structure

The curriculum is ten modules, totalling thirty teaching hours. Each module lasts roughly three hours and assumes the learner has finished the module before, except Module 1, which needs no prior knowledge. The sequence is deliberate: we introduce the smallest possible program first, add ways to store and display information, then add the two great ideas of choosing and repeating, then group steps into reusable functions, then organise many pieces of data, then read and write files, then spend a whole module learning to read errors, and finally combine everything into a finished mini-project. Debugging is threaded through every module, but Module 9 gives it dedicated time because it deserves it.

Figure 1: Curriculum structure — 10 modules building to a capstone



Reading note: each module builds on the one before it. Spread through M9 (debugging) and practise it inside every later module. M10 is the capstone — one useful small program.

Figure 1: Curriculum structure flowchart: 10 modules building to a capstone. *Each module assumes the one before it; M10 joins everything into one useful program.*

Image description: Flowchart with ten boxes in a snake layout: M1 computational thinking to M10 final mini-project. M9 debugging feeds M10.

Module	Hours	Prerequisites	Why it matters (one line)
M1 Computational thinking and problem decomposition	3	None	Programs start as clear steps, not code.
M2 Variables and data types	3	M1	Learners learn to name and store information safely.
M3 Input and output	2	M2	A program must talk to the user to be useful.
M4 Conditionals (if / else)	3	M3	Choosing is the first real logic.
M5 Loops (repeat)	3	M4	Repeating is how computers save us time.
M6 Functions (reusable steps)	3	M5	Grouping steps makes big tasks manageable.
M7 Lists and dictionaries	3	M6	Many real tasks are about organised collections.
M8 Files and simple data	3	M7	Programs become useful when they remember.
M9 Debugging and reading errors	3	M1–M8	Errors are information; reading them is a skill.
M10 Final mini-project (capstone)	4	M1–M9	One finished program proves real learning.

A few notes on the choices. Module 1 is unplugged for half its time: learners order steps for everyday tasks and trace a simple loop with cards, before seeing any Python. This mirrors the Code.org approach of unplugged first [4]

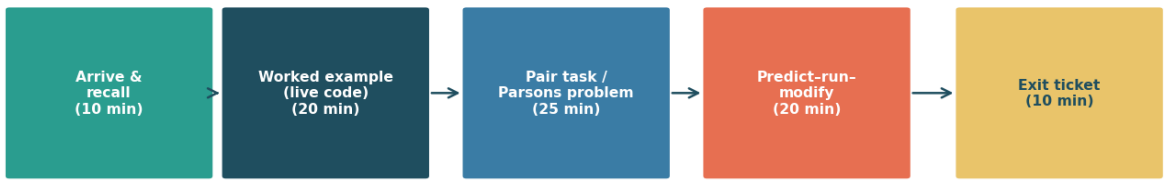
and lowers anxiety because no one can “break” paper. Module 2 introduces variables with a language caution — a variable is a named reference, not a box — and tests it with predicting tasks. Module 3 is short because showing output early gives a quick win. Modules 4 and 5 are the core logic pair; we give each a full three hours and use Parsons problems to practise ordering before free writing [5][6]. Module 6 introduces functions as reusable steps, which sets up the capstone. Module 7 covers lists (ordered collections) and dictionaries (name-to-value pairs) with everyday examples such as shopping lists and contact lists. Module 8 moves to files, so learners can read a list of names and write a summary, the kind of boring task that motivates adults [3]. Module 9 is not a break; it is a skill module where learners build an error log, read Python messages line by line, and practise fixing small bugs in almost-correct code. Module 10 is the capstone, where each learner picks a small, personal task and builds it with tutor support, then explains it in a short demonstration.

Hours add up to 30 (3+3+2+3+3+3+3+3+3+4). That total matters for resourcing: Section 8 prices people time against these hours, so the teaching plan and the budget use the same numbers.

6 Instructional methodologies and teaching methods

A typical session shows the research in action. We plan ninety minutes, split as roughly one-third instruction and two-thirds practice. The shape is the same every week so learners can settle into a rhythm: arrive and recall, watch a worked example with live coding, do a pair task or Parsons problem, run a predict–run–modify activity, and finish with an exit ticket. The consistency is deliberate; new learners spend enough energy on the ideas, not on figuring out the format.

Figure 4: What one typical 90-minute session looks like



Roughly one-third instruction, two-thirds practice. That ratio is deliberate — beginners learn by typing, not by watching.

Figure 4: Session structure diagram: what one 90-minute lesson looks like. *Short teaching, long practice, quick feedback — that ratio is the engine of the course.*

Image description: Five boxes in a row: Arrive and recall 10 minutes, Worked example 20 minutes, Pair task 25 minutes, Predict-run-modify 20 minutes, Exit ticket 10 minutes.

Part	Minutes	What happens	Why
Arrive & recall	10	Quick recap question, pair chat	Connects today to last time
Worked example	20	Instructor shows code live, thinks aloud	Models expert thinking [7]
Pair task / Parsons	25	Order or complete scrambled lines in pairs	Low-load practice before writing [5][6]
Predict–run–modify	20	Guess output, run, change one thing	Makes thinking visible
Exit ticket	10	One-question check, confidence mark	Early warning for tutors

Four methods carry the course. First, worked examples with live coding: the instructor solves a small problem while narrating each decision, including mistakes, so learners see how to recover. Second, Parsons problems: learners receive scrambled lines and must order them into a working program, which practises logic without the pressure of typing every character [5][8]. Third, pair programming: one learner types, one talks and checks, swapping every few minutes, which builds confidence and catches slips early [9]. Fourth, a predict–run–investigate–modify–make progression: learners predict, run, investigate the result, modify one element, and finally make a small change of their own. Unplugged activities return in Modules 1 and 2 to anchor new ideas in paper before code. Low-floor tasks — tasks with one easy start that many learners can attempt — open each practice set, so every learner has a success in the first ten minutes.

Tools are chosen for zero setup. Learners use Google Colab or a browser-based editor so nothing needs installing. Slides are shared as accessible PDFs with large type. Code starters are provided as Colab links, not attachments to download. Breakout rooms are used for pair work, with the instructor dropping in rather than hovering. Attendance and participation are tracked by exit tickets, not by a strict register, because we value presence in the work over presence in a seat.

7 Assessment techniques

We assess often and lightly, then once deeply. Formative assessment means small checks that inform teaching while learners can still change course. Summative assessment means a final judgment on the capstone. Both matter; the plan uses the first to keep learners moving and the second to show what they can now do. Feedback is specific, kind, and tied to the language of the learning outcomes, so a learner hears not “good effort” but “you used a conditional correctly; now check the alternative branch”.

Figure 5: Assessment map — every outcome is tested somewhere



Figure 5: Assessment map: which outcomes are tested where. *Every outcome is checked in at least two places, so one bad day cannot fail a learner.*

Image description: Matrix with 7 learning outcomes as rows and 4 assessments as columns: quizzes, exit tickets, small exercises, capstone. Cells marked tested or dash.

- Short quizzes (10 minutes, weekly): multiple choice and “predict the output” questions mapped to LO1–LO6. Immediate feedback.
- Exit tickets (every session, 5 minutes): one question plus a confidence mark. Tutors review same day and adjust the next session.
- Small exercises (after each module): Parsons problems, debugging tasks, and short writing tasks. Marked with a simple checklist.

- Capstone (M10): a finished mini-project plus a short live explanation. This is the only summative task.

The capstone rubric rewards process as well as product. Four criteria, each scored 1 to 4: correctness (does it run and give the intended result), use of ideas (are variables, a conditional, a loop, and a function used sensibly), readability (could another person follow it), and explanation (can the learner describe what it does and what they would change next). A total of 12 out of 16 is a pass, with the condition that correctness is at least 2, so a learner cannot pass on neatness alone. Resubmission of one section is allowed after feedback, because fixing a program is itself a learning outcome (LO6).

How we know the programme worked is defined in Section 11, but in brief: completion of the capstone by most learners, improvement from first to last exit ticket confidence, and the ability to explain the capstone without reading code aloud. Formative data drives weekly tweaks; summative data drives the next cohort plan.

8 Resource requirements and allocation

We plan with free tools and people time, so the budget is mostly about hours, not software. The numbers in this section use the same 30 teaching hours as Section 5 and the same 120 planned effort hours as Section 9, so the plan is internally consistent. The team is small on purpose: one lead instructor, two tutors, learner support, a pilot reviewer, and a coordinator. The venue is virtual, which removes room hire and makes recordings available for learners who need to replay.

Figure 6: Planned effort by role (120 hours total)

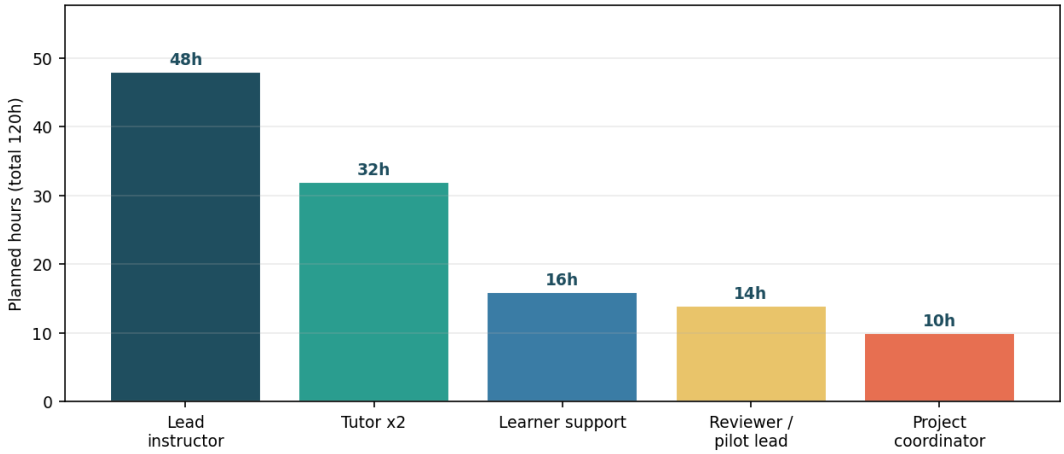


Figure 6: Resource allocation chart: planned hours by role (120 hours total). *Most time goes to teaching and tutoring, not administration.*

Image description: Bar chart showing hours by role: lead instructor 48, tutors 32, learner support 16, reviewer 14, coordinator 10.

Role	Planned hours	Main duties
Lead instructor	48	Teach 30h, prepare materials, lead feedback
Tutor x2	32	Support pairs, mark exercises, run help clinics
Learner support	16	Accessibility, reminders, hardship check-ins
Reviewer / pilot lead	14	Review materials, run pilot, log changes
Project coordinator	10	Schedule, reports, liaison

Tool and material costs are low. The platform is a free virtual meeting tool and Colab. Materials are the slides, handouts, and starter notebooks, all produced in-house and shared as PDFs. The only likely outlay is captioning or transcription support for the pilot recordings and printing for any in-person pilot, which we keep small and optional.

Item	Cost basis	Planned cost (₹)
Virtual platform (free tier)	Free	0
Colab / browser editor	Free	0
Slides, handouts, notebooks	In-house production	0
Captioning / transcription (pilot)	10 recordings est.	4000
Printing for pilot (optional)	Small pack	1500
People time (valued, not cash in internship)	120h	—
Contingency (5%)	On above cash items	275

Accessibility is budgeted as attention, not a line item: captioned videos, transcripts, high-contrast slides, keyboard-friendly editors, and extra support time for learners who need it. If a learner needs personal assistive technology, we coordinate with them individually rather than pretend a single solution fits all. The plan fits a virtual internship in Ahmedabad because no venue is needed and all tools run in a browser.

9 Timeline, milestones and deadlines

The dates in this section describe a planned future project, not events that have already happened. All dates are labelled planned. The baseline (kick-off) date is 5 November 2026 (planned), the first working day on or after the report date of 4 November 2026. We skip weekends and govind jayanti on 25 November 2026, which falls between the pilot and revision, so the revision phase starts after it. The assumption behind the baseline is stated in the Appendix so that a reader can adjust it if the project start moves.

Six phases span from discovery to launch. Discovery checks what learners need. Design turns that into outcomes and module outlines. Content creation writes the actual lessons. Review and pilot tests them with a small group. Revision fixes what the pilot shows. Launch prepares the real cohort. Each phase ends with a milestone that has a clear deliverable and owner, so progress can be checked without argument. Effort totals 120 planned hours, which matches Section 8 (48 + 32 + 16 + 14 + 10 = 120).

Figure 3: Planned curriculum-development timeline (Gantt) with milestones

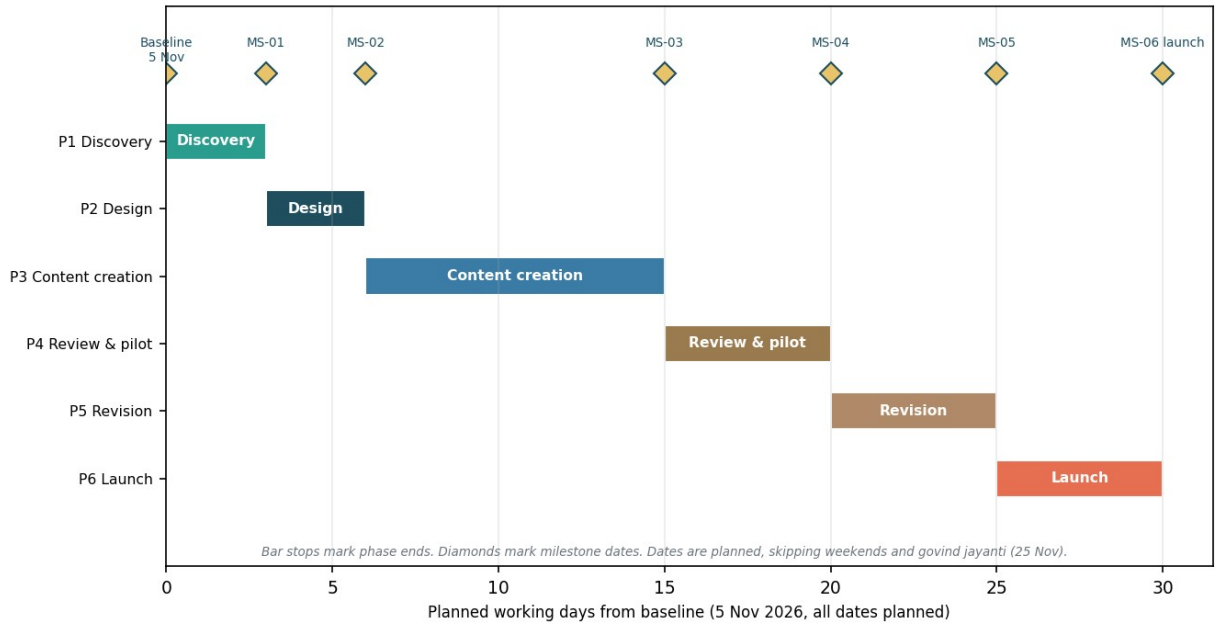


Figure 3: Planned timeline (Gantt) with milestones marked. *Six phases run in order; the pilot fits before revision, not at the end.*

Image description: Gantt chart with six horizontal bars for discovery, design, content creation, review and pilot, revision, launch. Diamond markers show six milestones.

ID	Milestone (planned)	Deliverable	Owner	Start (planned)	Due date (planned)	Dependency
MS-01	Kick-off & discovery sign-off	Learner needs note	Coordinator	5 November 2026	9 November 2026	—
MS-02	Design approved	Outcomes & module outlines	Lead instructor	10 November 2026	13 November 2026	MS-01
MS-03	Content draft complete	Slides, notebooks, handouts	Lead + tutors	16 November 2026	20 November 2026	MS-02
MS-04	Pilot reviewed	Pilot report & change log	Reviewer	23 November 2026	24 November 2026	MS-03
MS-05	Revision complete	Revised packs (v2)	Lead + tutors	26 November 2026	27 November 2026	MS-04
MS-06	Launch ready	Final packs, timetable, support rota	Coordinator	30 November 2026	4 December 2026	MS-05

Check the logic, not just the dates. MS-01 cannot start before the baseline. MS-02 depends on MS-01 because design needs the needs note. No milestone is due before its dependency, and due dates rise in order: 9, 13, 20, 24, 27 November, then 4 December 2026 (all planned). The holiday on 25 November is skipped, which is why revision starts on 26 November. If a phase slips by more than two working days, the launch date moves; a short weekly check-in in the shared chat is the agreed way to flag a slip early, rather than a heavy tracking system.

10 Risks, assumptions and mitigations

A plan that assumes nothing can go wrong is not a plan. The six risks below are the ones that could genuinely affect beginners or the team. Each is rated for likelihood (how likely it is) and impact (how much it would matter if it happened), with a practical mitigation that matches the rating. Ratings are judgments for planning, not predictions, and we revisit them at the pilot review.

Risk	Likelihood	Impact	Mitigation
R1. Learners drop out after first error	High	High	Worked examples, error log, low-floor tasks; exit tickets to spot struggling learners early
R2. Internet or Colab access fails	Medium	High	Browser-only design plus offline handouts; tutors keep a fallback notebook link
R3. Pair mismatch slows one learner	Medium	Medium	Rotate pairs, timed role swaps, tutor check-ins in breakout rooms
R4. Captions / accessible materials late	Medium	Medium	Write transcripts during production, not after; check contrast before pilot
R5. Pilot reveals module order is off	Low	High	Pilot in week before revision (MS-04) with time to reorder one module if needed
R6. Holiday or travel disrupts a week	Medium	Medium	Baseline skips 25 Nov 2026; launch can shift by two days without breaking the cohort

Assumptions are stated alongside risks because many risks become manageable once the assumption is clear. We assume 20 learners per cohort, 1 lead instructor plus 2 tutors, free tools, and that the pilot group is representative enough to test the order of modules. If any assumption fails — for example, if the cohort is 40 — the mitigation is to add a second tutor rather than dilute feedback. The ratings above assume the mitigations are in place; without them, R1 and R2 would be higher.

11 Success metrics and evaluation of the programme itself

We judge the programme by learner learning, not by how busy the team was. Four measures are collected, all defined before launch so we do not choose easy numbers afterwards. First, completion: the proportion of starters who submit a capstone. We aim for at least 80%. Second, learning: average quiz score in the final module compared with the first module. A clear rise suggests the sequence works. Third, confidence: the average exit ticket confidence mark at the end compared with the start. Fourth, explanation: the capstone rubric, where a pass needs both a reasonable total score and at least a basic correctness score. These four together tell a fair story: did learners stay, did they learn, did they feel able, and can they show it?

Evaluation of the programme itself happens after the pilot and after the first real cohort. The pilot uses the same measures on a small group plus tutor notes on where learners paused or paired well. Revision (Section 9, MS-05) must respond to pilot data in the change log; if a task was too hard, it is simplified, if too easy, it is extended. After the first cohort, a short feedback form asks learners what helped and what was confusing, and those answers feed the next plan. We also track overall effort against the 120 planned hours: if the team consistently exceeds the plan, the budget or staffing is revised rather than quietly accepting overwork.

12 Conclusion and next steps

This plan sets out a clear, warm, and realistic route from “never coded” to “I built something” for complete beginners. The decisions have reasons: Python because it reads clearly and runs in a browser, ten modules because thirty hours can be taught well, pairs and Parsons problems because research and classroom practice both show they lower the load, and frequent small checks because confidence matters. The timeline is planned, with a baseline of 5 November 2026 (planned), six phases, and six milestones that rise in order and skip the 25 November 2026 holiday, and the effort of 120 hours matches the resource table. Risks are named and rated, with mitigations that fit the ratings.

Next steps are short and owned. First, the coordinator holds the kick-off on 5 November 2026 (planned) and shares the needs-note template for MS-01. Second, the lead instructor drafts the design pack for MS-02 by 13 November 2026 (planned) using Section 4 outcomes. Third, tutors and the lead build the content pack for MS-03 by 20 November 2026 (planned). Fourth, the pilot runs and is reviewed by 24 November 2026 (planned), revision is done by 27 November 2026 (planned), and the launch pack is ready by 4 December 2026 (planned). Each step has a deliverable and an owner, so the project can start without waiting for further planning.

Glossary

Plain-language definitions of every technical term used in this plan. If a word appears fewer than a few times, it is still included if a non-technical reader might pause on it.

Term	Plain-language definition
Algorithm	A step-by-step set of instructions that solves a task.
Browser editor	A place to write code that runs inside a web browser, with nothing to install.
Capstone	The final mini-project where learners combine ideas into one finished program.
Colab	Google Colab, a free browser tool for running Python notebooks.
Conditional	A choice in a program, usually if / else, that runs different steps

Term	Plain-language definition
	depending on the situation.
Debugging	Finding and fixing mistakes in a program.
Dictionary	A collection that pairs each name with a value, like a contacts list.
Error message	The computer's note that explains what it did not understand or what went wrong.
Formative assessment	Small checks during learning that help teachers adjust teaching.
Function	A named, reusable set of steps that does one job.
Input / output	How information travels into a program and how results travel out.
List	An ordered collection of items, like a shopping list.
Loop	A structure that repeats steps until a condition changes.
Module	One teaching unit, about three hours, that builds on the previous one.
Parsons problem	A task where learners order scrambled lines of code instead of writing from scratch.
Programme	The whole course; we use module for each part inside it.
Python	The teaching language; it reads clearly and runs in a browser.
Syntax error	A mistake in the shape of a code sentence, so the computer cannot parse it.
Variable	A named place that holds a value that can change, like a label on a shelf.

References

Sources are listed in the order they are first cited. All were found through public web pages during preparation on 4 November 2026. Paraphrased text is original; no passage is copied.

No.	Source (title, author where relevant, URL)
[1]	CS50x: Harvard CS50 Introduction to Computer Science. https://cs50.harvard.edu/x/
[2]	Severance, C. Python for Everybody (PY4E). https://www.py4e.com/ and https://do1.dr-chuck.com/pythonlearn/EN_us/pythonlearn.pdf
[3]	Sweigart, A. Automate the Boring Stuff with Python (2nd ed., free online). https://automatetheboringstuff.com/2e/
[4]	Code.org. Computer Science Fundamentals Curriculum. https://code.org/en-US/curriculum/computer-science-fundamentals
[5]	Raspberry Pi Foundation. Supporting learners with programming tasks through Parsons problems (2024). https://www.raspberrypi.org/blog/supporting-learners-with-programming-tasks-through-ai-generated-parsons-problems/
[6]	Teach Computing. Parsons problems — Pedagogy Quick Read QR13. https://static.teachcomputing.org/pedagogy/QR13-Parsons-problems.pdf
[7]	MIT Teaching and Learning Lab. Worked examples. https://tll.mit.edu/teaching-resources/how-people-learn/worked-examples/
[8]	Kapor Foundation. Parsons Problem strategy guide.

No.	Source (title, author where relevant, URL)
	https://kaporfoundation.org/strategies_guide/parsons-problem/
[9]	Pair programming reviews for novices: Lui & Yap (2006); Denner et al. (2014). Abstract-level findings only; no effect sizes quoted. [verify before citing: exact effect sizes]
[10]	Worked examples and cognitive load: ACM, Effects of worked examples with explanation types (2025). https://dl.acm.org/doi/full/10.1145/3 732 791
[11]	iD Tech. Scratch vs Python: which is right for beginners? https://www.idtech.com/blog/scratch-vs-python
[12]	The Python Coding Book. Python readability, PEP 8 style guide. https://thepythoncodingbook.com/2021/10/11/python-readability-the-pep-8-style-guide-and-learning-latin/

Citation note: this plan quotes no statistics, dates of studies, or direct quotations other than titles and URLs above. Where a source was available to me only as a search result snippet or abstract, I used only its general finding and marked the entry accordingly in the References table.

Appendix: assumptions and decisions log

Baseline and anchoring facts used throughout this plan: offer received 2 November 2026; internship period 2 November 2026 to 30 November 2026; report prepared on 4 November 2026; planned baseline (kick-off) 5 November 2026 (planned), first working day on or after the report date. All Section 9 milestone dates are planned, fall on or after the baseline, rise in order, skip weekends, and skip govind jayanti on 25 November 2026. No milestone is listed as completed except the offer and this report itself.

ID	Assumption / Decision	Reason
A1	20 learners per cohort	Keeps pair work and tutor feedback feasible
A2	1 lead + 2 tutors, 120 planned hours	Matches Section 8 budget
A3	Baseline 5 November 2026 (planned)	First working day after report date 4 November 2026
A4	Free tiers for tools; cash only for captions and pilot printing	Keeps budget modest and realistic
A5	Accessibility: captions, transcripts, contrast checks	Design requirement, not an add-on
D1	Teaching language: Python	Readable, browser-based, widely supported [2][3][11][12]
D2	Virtual delivery via Colab / browser editor	No installs, fits remote internship
D3	10 modules, 30 teaching hours	Scope fits one cohort with a capstone

Author: Meghana Krishnakumar Nair. This plan was prepared for Week 1 of the Virtual Programming Fundamentals Internship in Ahmedabad. Figures were generated with matplotlib in a consistent palette and saved as PNGs embedded in this document. The build is reproducible from the repository with the commands in the README.